

Agentes heurísticos y modelos de lenguaje para PyCatan

Shiyi Cheng

Abril de 2026

1. Objetivo

El objetivo de la práctica es diseñar, implementar y evaluar un agente inteligente para el simulador PyCatan. El trabajo se divide en dos partes. La primera desarrolla un agente heurístico clásico y lo compara con agentes *random* y agentes estándar. La segunda integra modelos de lenguaje (LLM) en decisiones concretas del agente para estudiar si son viables como política de decisión dentro de una simulación multiagente.

La aportación principal es doble: por un lado, un agente heurístico incremental con rendimiento claramente superior al aleatorio; por otro, una infraestructura experimental para comparar proveedores LLM, medir *fallback*, latencia y calidad de decisión, y separar los resultados válidos de aquellos contaminados por decisiones de respaldo.

2. Diseño del agente heurístico

El agente heurístico se construyó de forma incremental, activando mejoras por bloques de decisión. Esta estructura permitió medir qué parte del comportamiento aporta más rendimiento. Las decisiones principales cubiertas son:

- **Colocación inicial:** evaluación de nodos por producción esperada, diversidad de recursos y calidad de la carretera inicial.
- **Construcción:** priorización dinámica de ciudad, pueblo, carretera y carta de desarrollo según recursos disponibles y estado del tablero.
- **Comercio:** uso de intercambios con la banca cuando permiten completar recursos críticos para el objetivo actual.
- **Ladrón y cartas:** reglas simples para no desperdiciar acciones y mantener una política estable.

La heurística de nodos combina la probabilidad del dado con la variedad de recursos. En Catan, producir de forma regular y no depender de un único recurso suele ser más importante que maximizar una casilla aislada. Para la construcción, el agente intenta convertir recursos en puntos de victoria de forma temprana: primero ciudades si existen pueblos valiosos, después pueblos, carreteras y cartas. La carretera se valora por su capacidad de abrir nuevos nodos útiles.

3. Infraestructura experimental

Se refactorizaron los benchmarks en un módulo común, `benchmark_common.py`, con wrappers para `vs_random` y `vs_standard`. La infraestructura genera CSV homogéneos con partidas, victorias, *winrate*, puntos medios, rango medio, decisiones LLM, *fallback* y proveedor/modelo cuando aplica.

Los experimentos heurísticos se ejecutaron con semillas reproducibles. En la evaluación iterativa se usaron las semillas 42, 31415 y 2026, 25 partidas por posición contra random, es decir, 100 partidas por semilla. Para los experimentos LLM se utilizaron ejecuciones completas cuando el coste lo permitió y ejecuciones ligeras para los modelos locales.

Para facilitar la reproducibilidad, los resultados finales se guardaron en CSV dentro de `PyCatan/results`. Los experimentos heurísticos finales se encuentran en `results/heuristic_final`; los experimentos LLM principales están en `results/bedrock_llm`, `results/upv_llm`, `results/llm_phase2` y `results/llm_light`. Las ejecuciones fijan semillas, número de partidas y máximo de rondas mediante variables de entorno, por lo que pueden repetirse sin cambiar el código del agente.

4. Resultados del agente heurístico

La Tabla 1 resume el experimento de ablación. La columna *gate 70%* indica si la configuración supera el requisito mínimo frente a agentes random.

Iteración	Media winrate	Desv. típica	Mínimo	Gate 70 %
iter0 random baseline	0.2733	0.0047	0.2700	No
iter1 game start	0.2800	0.0294	0.2500	No
iter2 build phase	0.8433	0.0499	0.7900	Sí
iter3 commerce trade	1.0000	0.0000	1.0000	Sí
iter4 thief devcards	1.0000	0.0000	1.0000	Sí

Cuadro 1: Resumen del pipeline iterativo frente a agentes random. Fuente: `results/iterative/benchmark_iterative_summary.csv`.

El salto más importante aparece al activar `on_build_phase`. Esto confirma que, en este simulador, convertir recursos en construcciones de forma coherente es más decisivo que mejorar únicamente la colocación inicial. La fase de comercio termina de cerrar el comportamiento: al desbloquear intercambios orientados a objetivos, el agente alcanza 100 % de victorias contra random en las semillas del gate.

Frente a agentes estándar, la primera versión competitiva evaluada fue `iter2_build_phase`. Sus resultados aparecen en la Tabla 2. Aunque ya supera ampliamente al random, el rendimiento frente a agentes más fuertes todavía no es dominante, lo que muestra que el comercio y los módulos posteriores no son cosméticos.

Seed	Partidas	Victorias	Winrate	Puntos medios
42	500	203	0.4060	6.5140
2026	500	180	0.3600	6.2820
31415	500	209	0.4180	6.5680

Cuadro 2: Iteración 2 frente a agentes estándar. Fuente: CSV de la iteración 2 en `results/iterative`.

Finalmente se ejecutó el agente heurístico completo, ya con todas las mejoras activadas, frente a random y frente a agentes estándar. La Tabla 3 resume esos resultados. Cada fila contiene 100 partidas por seed.

Benchmark	Seeds	Partidas	Victorias	Winrate medio	Puntos medios
vs random	42, 2026, 31415	300	300	1.0000	10.1133
vs estándar	42, 2026, 31415	300	271	0.9033	9.7200

Cuadro 3: Evaluación final de `ShiyiHeuristicAgent`. Fuentes: `results/heuristic_final/vs_random.csv` y `results/heuristic_final/vs_standard.csv`.

Este resultado refuerza la conclusión experimental: el agente no solo explota la debilidad del random, sino que mantiene un rendimiento alto frente a agentes estándar del simulador. El paso de 0.40 aproximadamente en `iter2` a 0.90 en la versión final muestra que comercio, ladrón y cartas sí aportan robustez contra oponentes no aleatorios.

5. Integración de LLM

El agente LLM se implementó como una variante de `ShiyiHeuristicAgent`. El modelo se consulta en dos decisiones: `on_game_start` y `on_build_phase`. En cada llamada se envía el estado mínimo necesario, una lista indexada de acciones legales y una instrucción para devolver JSON con el campo `action_index`. Si la respuesta es inválida, fuera de rango o no llega a tiempo, el sistema registra *fallback* y usa la acción heurística.

Se diseñaron tres variantes de prompt: `compact_json`, `resource_focus` y `safe_legal`. Los experimentos finales se concentraron en `compact_json`, porque es el formato más corto y el que mejor encaja con el requisito de respuesta estructurada. Durante las pruebas locales se detectó un fallo importante: algunos modelos pequeños tendían a copiar el JSON de entrada o a confundir `action_index` con `node_id`. Para corregirlo, el prompt pasó a usar acciones explícitamente indexadas y la restricción de que `action_index` no es ni `node_id`, ni `road_to`, ni tipo de construcción.

Además de diseñarlos, se hizo una comparación ligera de prompts con Bedrock `nova-micro-v1`. La Tabla 4 muestra que los tres prompts consiguen ganar las partidas del ensayo, pero difieren en robustez de formato. En este caso, `safe_legal` fue el más estable porque no produjo *fallback*.

Prompt	Partidas	Winrate	Puntos medios	Fallback
compact_json	4	1.0000	10.25	0.0629
resource_focus	4	1.0000	10.00	0.0432
safe_legal	4	1.0000	10.00	0.0000

Cuadro 4: Comparación ligera de prompts con Bedrock nova-micro-v1. Fuente: `results/llm_phase2/llm_phase2_summary.csv`.

Esta prueba no pretende demostrar superioridad estratégica con solo cuatro partidas, pero sí sirve para evaluar el formato de interacción. En LLM aplicados a simuladores, un prompt que gana pero genera *fallback* es menos fiable que uno ligeramente más conservador y estable.

6. Resultados LLM

La Tabla 5 recoge los resultados comparables disponibles. Se marca como inválido cualquier resultado con *fallback* alto, porque en ese caso la política real no es el LLM, sino la heurística de respaldo.

Proveedor	Modelo	Benchmark	Prompt	Partidas	Winrate	Avg pts	Fallback	Uso
Bedrock	nova-lite-v1	random	compact	100	1.0000	10.04	0.0000	válido
Bedrock	nova-lite-v1	standard	compact	100	0.9100	9.75	0.0000	válido
Bedrock	llama3.3-70b	random	compact	100	1.0000	10.12	0.0002	válido
Bedrock	llama3.3-70b	standard	compact	100	0.9400	9.77	0.0002	válido
UPV	llama-mini	random	compact	100	1.0000	10.05	0.0143	aceptable
UPV	llama-mini	standard	compact	100	0.9000	9.63	0.0000	válido
Ollama	qwen2.5:0.5b	random	compact	100	0.8300	9.21	0.0000	válido parcial
Ollama	qwen2.5:0.5b	standard	compact	4	0.2500	4.75	0.0000	smoke test

Cuadro 5: Resultados LLM principales. Fuentes: `results/bedrock_llm/20260422_093854`, `results/upv_llm/20260422_120019` y `results/llm_light/20260424_124431`.

También se obtuvieron resultados descartados. `mistral.mistral-7b-instruct-v0:2` en Bedrock registró *fallback_rate*=1.0, por lo que sus victorias no pueden atribuirse al modelo. En UPV, `phi4-mini:3.8b` presentó *fallback* de 0.4288 contra random y 0.5751 contra estándar, de modo que tampoco es una configuración fiable. En Ollama, una ejecución antigua de `gemma2:9b` produjo 0.9745 de *fallback*; después de corregir el prompt, las llamadas directas ya respondían JSON válido, pero no se completó un benchmark limpio. Estos descartes son importantes: si se ignora el *fallback*, el CSV puede mostrar victorias que realmente pertenecen al agente heurístico.

7. Análisis de modelos

Los modelos de Bedrock fueron los más estables. `nova-lite-v1` ofreció buen rendimiento sin *fallback* y `llama3.3-70b` obtuvo el mejor resultado frente a agentes estándar. UPV `llama-mini` también fue utilizable, aunque con un pequeño *fallback* contra random. En local se detectaron dos modelos mediante Ollama: `qwen2.5:0.5b` y `gemma2:9b`. El modelo pequeño, ejecutado en local con una AMD 6600 XT, consiguió completar 100 partidas contra random sin *fallback*, pero su rendimiento fue peor que Bedrock: 0.83 de *winrate* y 9.21 puntos medios. Esto sugiere que responder JSON válido no garantiza buena política.

La latencia local de `qwen2.5:0.5b` fue baja para un LLM: 488 ms medios por decisión en la ejecución completa contra random, con 8847 decisiones. Aun así, el coste total fue elevado porque se consulta el modelo muchas veces durante la partida. Por ello, una integración práctica debería reservar los LLM para decisiones de alto impacto o usarlos para generar heurísticas, no necesariamente para cada paso frecuente.

8. Viabilidad de los LLM en PyCatan

Los LLM pueden integrarse técnicamente en PyCatan, pero presentan tres riesgos. Primero, la salida estructurada no está garantizada: incluso con temperatura baja, algunos modelos devuelven índices fuera de rango o texto adicional. Segundo, la latencia multiplica el tiempo de benchmark. Tercero, si se permite *fallback* silencioso, se sobreestima la calidad del modelo.

La conclusión experimental es que los LLM grandes y bien servidos funcionan como política aproximada para decisiones acotadas, especialmente cuando reciben una lista de acciones legales. Sin embargo, para un agente competitivo, la heurística sigue siendo más barata, reproducible y fácil de depurar. El uso más razonable del LLM sería como apoyo en decisiones estratégicas poco frecuentes, generación de explicaciones o exploración de variantes heurísticas, no como sustituto completo de la política del agente.

9. Uso de herramientas de IA

Durante el desarrollo se usaron asistentes de programación basados en IA para refactorizar benchmarks, diseñar el pipeline experimental, implementar el motor LLM, depurar errores de formato JSON y redactar esta memoria. Su utilidad principal fue acelerar tareas de integración y análisis. La limitación más clara apareció precisamente en la experimentación LLM: una salida aparentemente correcta puede esconder *fallback*, por lo que fue necesario validar con CSV, métricas y pruebas manuales.

10. Conclusiones

1. El agente heurístico supera ampliamente el objetivo de 70 % frente a random desde la iteración de construcción, y alcanza 100 % al añadir comercio en las semillas evaluadas.
2. La versión final obtiene 100 % frente a random y 90.33 % frente a agentes estándar en 300 partidas por benchmark.
3. La ablación muestra que `on_build_phase` es la mejora más decisiva; el comercio y los módulos posteriores consolidan el rendimiento contra oponentes más fuertes.
4. La integración LLM funciona en dos decisiones reales del agente: colocación inicial y construcción.
5. Se compararon tres prompts. `safe_legal` fue el más robusto en el ensayo ligero porque consiguió *fallback*=0.
6. Bedrock `nova-lite-v1` y `llama3.3-70b` son las configuraciones más sólidas entre las evaluadas.
7. Ollama local es viable técnicamente, pero los modelos pequeños requieren prompts muy estrictos y aun así ofrecen menor calidad de decisión.
8. El *fallback* debe tratarse como métrica crítica. Los resultados con *fallback* alto no son válidos para comparar modelos.